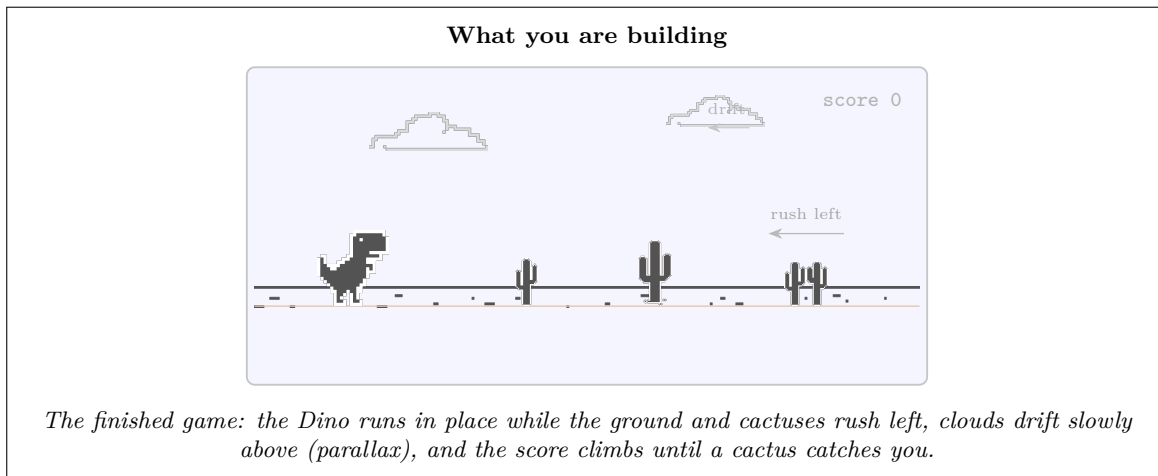


Teacher reference — the finished game, every script shown.

Dino Run - teacher reference. Building on a Dino that already runs (swapping costumes) and jumps in place, these notes carry the game to the finish in five pieces, each a small project with its complete scripts: a *moving ground* so the world rushes past, drifting *clouds* for depth, *cactuses* to jump over, a *score* that climbs while you survive, and finally a *broadcast* that ends the game on a crash. The order is deliberate - each piece sets up the next, and the score piece stops at a wall (a crash it cannot act on) that the broadcast piece knocks down. The endless ground is the one genuinely new trick, so it goes first; watch it move at cs17.org/ground-leapfrog-visualizer.html. Every script below is shown finished - assemble in the real Dino project on scratch.mit.edu.



The block groups you will use. Each block has a colour. The colour tells you which group it is in. Find the colour first, then find the block.

Motion	(blue)	Move the sprite: move, turn, go to, point towards, change x or y.
Looks	(purple)	Change how the sprite looks: say, costume, size, backdrop.
Sound	(pink)	Play a sound.
Events	(yellow)	Start a script (green flag, key, click, when I receive) and broadcast messages.
Control	(orange)	Choose when and how often blocks run: wait, forever, repeat, if, stop.
Sensing	(light blue)	Ask about the world: touching?, key pressed?, mouse, ask and answer.
Operators	(green)	Do maths and comparisons: plus, pick random, less-than, equals, more-than, and/or.
Variables	(dark orange)	Remember a value like score: set, change, and show a variable.

Project 1: The moving ground

Make the ground rush past forever. The trick: two ground tiles, each a little wider than the stage, chasing each other left. When one slides a full stage-width off the left, it leaps back to the right behind its partner - and because the tiles are a touch wider than the screen, they overlap, so there is never a gap. That endless belt is what makes the Dino feel like it is running.

Get ready. Draw or import a ground strip a little **wider than the stage** - make it **520 wide** (the stage is 480) - and name the sprite **Ground1**. It sits low, at y: -80. The extra 40 pixels are what let the two tiles overlap and hide the seam (see the note). You build Ground1's script, then duplicate it into Ground2.

New blocks.

`x position`

A blue Motion reporter: how far left or right the sprite's centre is. Near the right edge it is a big plus number; as the tile slides left it drops past -240 into the minuses.

`if <(x position) < (-480)>`

True once this tile has slid a full stage-width left - its partner now covers the whole screen, so this tile is free to leap.

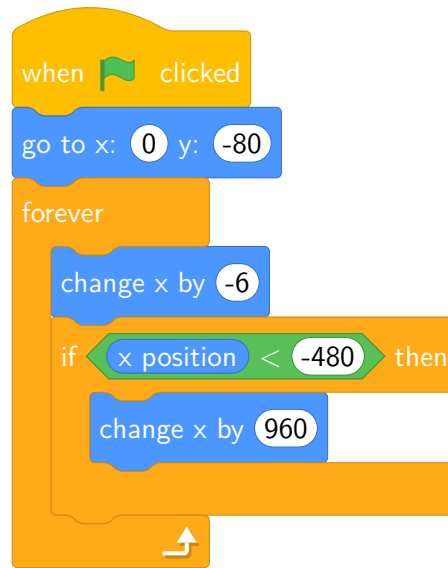
`change x by (960)`

Leap the tile two stage-widths to the right, landing it just behind its partner and keeping the pair 480 apart.

Build it.

1. On **Ground1**, start with `when green flag clicked`, then `go to x: 0 y: -80`. This tile fills the stage (and a touch more - it is 520 wide).
2. Add a `forever` loop. Its first block is given: `change x by (-6)`, which slides the tile left a little every tick.
3. Below it is an `if` shell with two `? your turn` gaps. The grey hexagon is the test: has this tile slid a *full stage-width* off the left? The grey block inside is the action: leap it back to the right, behind its partner.
4. Green flag. One tile scrolls left and leaves a blank gap behind it - that is expected, one tile cannot cover the screen and be sliding off at once. We need a second tile to fill that gap.
5. Right-click Ground1 and choose *duplicate*. Rename the copy **Ground2**. Change *only* its start block to `go to x: 480 y: -80` - everything else stays identical. Now while Ground1 fills the screen, Ground2 waits just off the right; they take turns forever, overlapping by 40 pixels so no seam ever shows.

Ground1's scroll loop. Fill the two grey gaps in the `if` - the test (grey hexagon) and the leap (grey block). Ground2 is this exact script with a start of x: 480:



Ground1: scroll left, then leap (Ground2 = same, start x: 480)

Why 520 wide and not exactly 480? Because **Scratch fences every sprite** - it never lets a sprite slide fully off the stage, always keeping about 15 pixels on screen. An exactly-480 tile's centre can therefore never reach -480, so `if <(x position) < (-480)>` would never be true and the belt would jam. Widening each tile to 520 fixes both halves: the wider box may legally slide until its centre passes -480 (so the test fires), and the 40-pixel overlap covers the hand-off - when a tile passes -480 its partner is already at x: 0 covering the screen, so the leap is invisible.

Why change x by (960) and not set x to (480)? Both put the tile back on the right, but only `change x` carries the fence overshoot forward, so the two tiles keep more overlap at every speed. One wrap at speed -50 - the step overshoots to -500 and the fence clamps it to -485:

tick	G1 (raw→fenced)	G2	change x→G1 (ovl)	set x→G1 (ovl)
9	-450	30	-450	-450
10	-500 → -485	-20	475 (25)	480 (20)
11	—	-70	425 (25)	430 (20)
12	—	-120	375 (25)	380 (20)

overlap = 520 - (G1 - G2); a gap opens only if it drops below 0. Here *both* stay safe - the 40-pixel cushion absorbs the drift - but `change x` keeps more: it carries the fence overshoot forward, `set x` throws it away. Prefer `change x` by (960) because it needs no fixed 480 and keeps working with more tiles, other widths, or a changing speed - not because `set x` visibly tears here (it does not, until very high speeds, and then both do).

Watch it at cs17.org/ground-leapfrog-visualizer.html.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Change both tiles' `change x by (-6)` together to set the running speed. Bigger negative = faster world.

Expected: The ground rushes past faster, so the game feels quicker.

2. Make one `speed` variable (For all sprites), `set [speed] to (6)` once, and use `change x by ((0) - (speed))` on both tiles. Later the cactus can read the same variable so ground and cactus always match.

Expected: One number controls how fast the whole world moves.

3. Add depth: a second, faraway backdrop layer (hills, clouds) on its own pair of tiles scrolling slower, say -3. Near things move faster than far things - that is parallax.

Expected: The background drifts slowly behind the fast ground, giving a sense of distance.

Project 2: Clouds drifting by (your first clones)

Fill the sky with clouds that drift slowly past, far behind the action. This is your first taste of clones: the Cloud sprite stays hidden and spawns copies of itself; each copy drifts across on its own and then deletes itself. No jumping, no score - clouds are just scenery, so it is the gentlest way to meet clones. The one trick that sells the depth: clouds move much slower than the ground.

Get ready. Add a **Cloud** sprite (*Choose a Sprite*, or paint a small puffy cloud) and name it **Cloud**. It gets two scripts: one that spawns clones, and one that each clone runs. Clouds never touch the Dino, so there is no collision to worry about.

New blocks.

<code>create clone of [myself]</code>	Make a live copy of this sprite. Each copy runs its own <code>when I start as a clone</code> script from the top.
<code>when I start as a clone</code>	A hat: the script under it runs on each new clone the instant it is born.
<code>delete this clone</code>	Remove this clone once it has drifted off screen, so clones do not pile up.
<code>pick random (2) to (5)</code>	An Operators reporter: a fresh random number each time - here the gap in seconds between clouds, so they never come evenly.

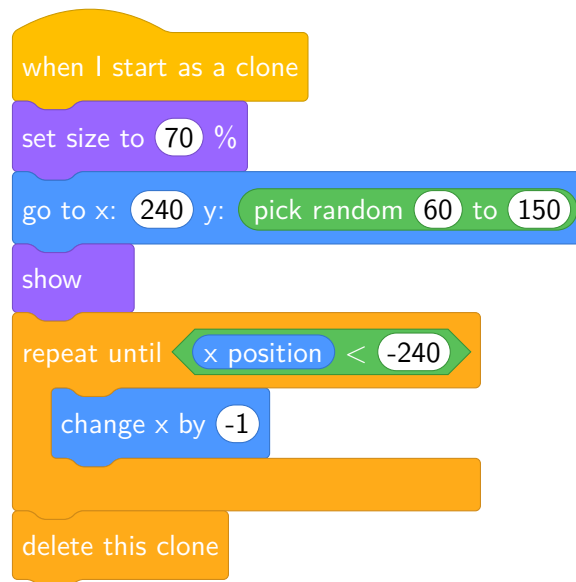
Build it.

1. First the **spawner**. On **Cloud**: `when green flag clicked`, then `hide` - the original never shows, it only makes clones. Add a `forever` whose first block is `wait (pick random (2) to (5)) seconds` (the random gap between clouds).
2. The next block is a `? your turn`: make a new cloud. (Which Control block creates a copy of this sprite?)
3. Now the **clone** script. Start it with `when I start as a clone`, then `set size to (70) %` (clouds are small - they are far away), `go to x: (240) y: (pick random (60) to (150))` (the right edge, at a random height up in the sky), and `show`.
4. Add a `repeat until` loop holding `change x by (-1)` - a slow drift, much slower than the ground's -6. The `? your turn` is the grey hexagon: stop once the cloud has drifted off the left edge.
5. After the loop, `delete this clone` is given. Green flag: clouds drift in from the right at random heights and gaps, slow and silent behind everything else.

The spawner, then the clone's drift loop. Fill the grey block (make a clone) and the grey hexagon (the off-left test):



Cloud (original): stay hidden, spawn a cloud now and then



Each cloud: drift slowly in from the right, then delete off the left

This is exactly the pattern the cactus will use next - a hidden spawner making clones, each running the same short script and then cleaning itself up with `delete this clone`. Clouds are the easy version: they cannot hurt you, so there is no *touching* test and no score - just drift and delete. The slow `change x by (-1)` is the whole point: the real Chrome game moves clouds at about a fifth of the ground speed, so near things (ground, cactus) rush by while far things (clouds) creep - that difference is called **parallax**, and it is what makes a flat screen feel deep.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Vary the size: change `set size to (70)` to `set size to (pick random (40) to (80))` so some clouds look nearer (bigger) and some farther (smaller).

Expected: Clouds come in different sizes, adding depth.

2. Give each cloud its own drift speed: at the top of the clone script pick a number into a `my drift` variable and use `change x by ((0) - (my drift))`, so some clouds crawl and some glide.

Expected: The sky feels alive - clouds move at their own paces.

3. Tie clouds to the game speed: use `change x by ((0) - (speed) / (5))` so when the game speeds up the clouds do too - but always a fifth as fast, just like Chrome.

Expected: The whole world speeds up together, keeping the depth.

Project 3: The cactuses

Send cactuses charging in from the right, two or three on screen at once, at random gaps - so the run never feels the same twice. It uses the same clone trick you just met with the clouds - a hidden sprite spawning copies that each run across and delete themselves - but now the copies sit on the ground and race at full speed, because you have to jump them.

Get ready. Add a **Cactus** sprite (*Choose a Sprite*: a cactus, a ball, anything spiky), make it small, and name it **Cactus**. You will give it two scripts: one that spawns clones, and one that each clone runs.

New blocks.

create clone of [myself]

Same as the clouds: make a live copy that runs its own **when I start as a clone** script.

when I start as a clone

Same hat as the clouds - runs on each new cactus clone the instant it is born.

delete this clone

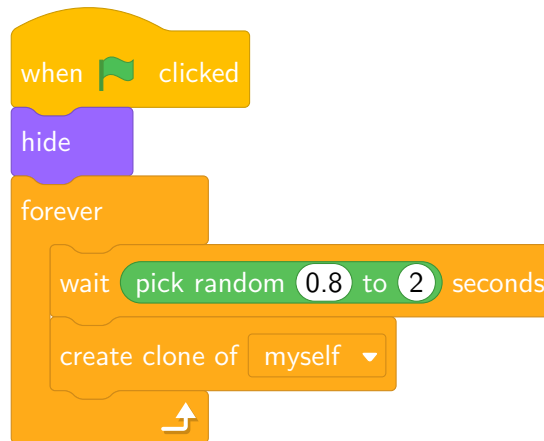
Same as the clouds: remove the clone once it has done its job.

repeat until <(x position) < (-250)>

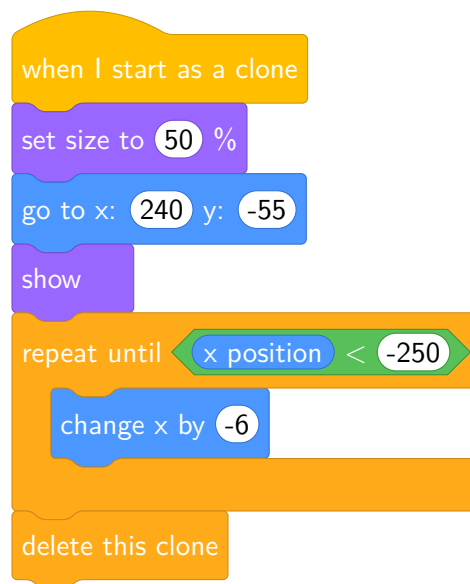
Keep the clone moving left until it has passed the left edge (x: -240); then stop and delete it.

Build it.

1. First the **spawner**. On **Cactus**: **when green flag clicked**, then **hide** - the original never shows, it only makes clones. Add a **forever** whose first block is **wait (pick random (0.8) to (2)) seconds** (the random gap between cactuses).
2. The next block is a **? your turn**: make a new cactus. (Which Control block creates a copy of this sprite?)
3. Now the **clone** script. Start it with **when I start as a clone**, then **set size to (50) %**, **go to x: (240) y: -55** (the right edge, on the ground), and **show**.
4. Add a **repeat until** loop holding **change x by (-6)** - the same speed as the ground. The **? your turn** is the grey hexagon: stop when the cactus has run off the left edge.
5. After the loop, **delete this clone** is given - the cactus is gone, so remove the copy.
6. Green flag. Cactuses stream in from the right, two or three at a time, at changing gaps. Jump them! The spawner, then the clone's run loop. Fill the grey block (make a clone) and the grey hexagon (the off-left test):



Cactus (original): stay hidden, spawn a clone now and then



Each clone: run in from the right, then delete off the left

This is the falling rock from Scratch F, turned sideways and cloned. One rock became many cactuses: the hidden original is a spawner, and every clone runs the same short script on its own, then cleans itself up with `delete this clone`. Because the wait is random, the gaps vary; because a clone takes a couple of seconds to cross, two or three are usually on screen together. Keep the cactus speed the same as the ground or it looks like it is skating.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Give the cactus 3 costumes and, right after `show`, add `switch costume to (pick random (1) to (3))`. Now each clone looks a little different.

Expected: Cactuses vary in look instead of all being identical.

2. Random speed per cactus: at the top of the clone script pick a speed once into a `my speed` variable and use it in the loop, so each clone runs at its own pace.

Expected: Some cactuses rush, some crawl - the timing is unpredictable.

3. Ramp it up: shorten the spawn wait as the game goes on (a smaller `pick random` range) so cactuses come thicker the longer you survive.

Expected: The screen gets busier the further you get.

Project 4: Score, and spotting the crash

Two things here, and a cliffhanger. First a score that climbs while you survive - exactly like the real Chrome game, which counts how far you have run. Second, the cactus learns to spot the instant it hits the Dino. But spotting a crash and acting on it are different: a cactus clone can only control itself - it cannot make the Dino react or stop the game. So the crash is detected and then...nothing happens yet. That wall is exactly what the next piece knocks down.

Get ready. Make a variable **score** For all sprites. You add one scoring script to the **Dino**, and one **if** to the **cactus clone**.

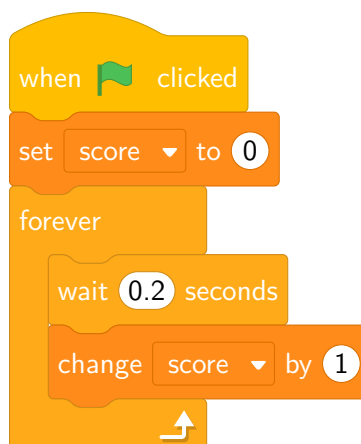
New blocks.

<code>change [score] by (1)</code>	Add one to the score. The Dino does this over and over while you are alive, so the score measures how long you have survived - Chrome scores the same way (distance run, not cactuses dodged).
<code>if <touching [Dino]?></code>	Checked on a cactus clone: true the moment this cactus hits the Dino - the crash. For now its inside is empty: detecting the hit is easy, acting on it is the missing piece.

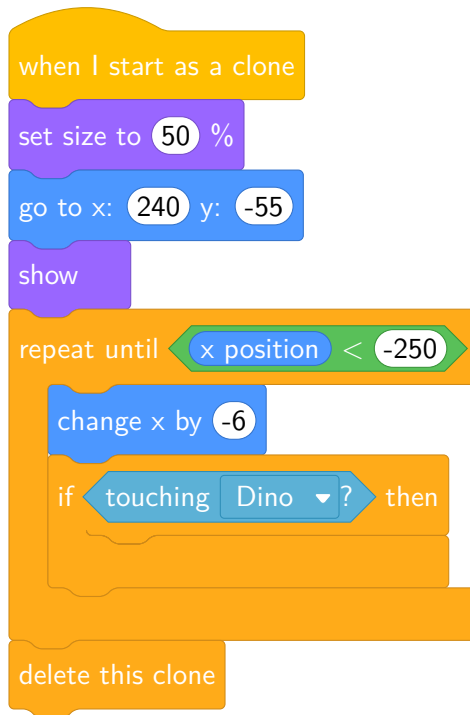
Build it.

1. The score lives on the **Dino**: `when green flag clicked`, `set [score] to (0)`, then a `forever` with `wait (0.2) seconds` and `change [score] by (1)`. While you are alive the score climbs.
2. Spotting the crash. Open the **cactus clone** script. Inside the `repeat until` loop, under `change x by (-6)`, add `if <touching [Dino]?>`. The cactus can now *see* the moment it hits the Dino.
3. But leave that `if` empty - and that is the point. What should happen on a crash? The game should end, the Dino should react, everything should stop. Yet a cactus clone can only move and delete *itself*; it has no way to reach the Dino or the score tick. Run it: touch a cactus and...nothing. We have hit the wall.
4. That wall is what the next piece knocks down: a `broadcast` - one message every sprite hears at once - so a single crash can end the game for everybody.

The Dino's score tick, and the cactus that now spots a crash - its `if` is deliberately empty, waiting for the next piece:



Dino: start at zero, then tick the score up while alive



Cactus clone: run across and spot a hit - but it cannot act on it yet

The score is the easy half: the Dino counts time while alive, exactly how Chrome scores (distance, not cactuses dodged). The crash is the interesting half - and it stops mid-air on purpose. A clone is an island: it can change its own x and delete itself, and that is about all. It cannot tell the Dino to show Game Over, cannot stop the score tick, cannot reach the clouds. Detecting the crash is easy; getting that news to everyone is the missing tool - and that is the whole reason the next piece, **broadcast**, exists.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Make the score climb faster as the game speeds up: use **change [score] by (speed)** instead of by 1 (once you have a **speed** variable).

Expected: Score rises quicker the faster the world moves - long runs are rewarded.

2. Show the score on the stage: tick the checkbox next to the **score** variable so the number sits in the corner and climbs as you play.

Expected: The live score is visible on the stage.

Project 5: Game over: the broadcast

Knock down the wall from the last piece. The cactus can spot a crash but cannot end the game alone. The fix is **broadcast**: it sends one message to the whole project, and every **when I receive** - on any sprite - runs at once. So the cactus shouts game over, and the Dino hears it and reacts. One signal, many sprites acting together - the one thing a lone sprite could never do.

Get ready. No new sprites. You finish the empty **if** on the **cactus clone**, and add one **when I receive** script to the **Dino**.

New blocks.

`broadcast [game over]`

Send the message *game over* to the whole project, then carry on - it does not wait. Make the message with the dropdown: *New message*.

`when I receive [game over]`

A hat: the script under it runs the instant *game over* is broadcast - on this sprite or any other.

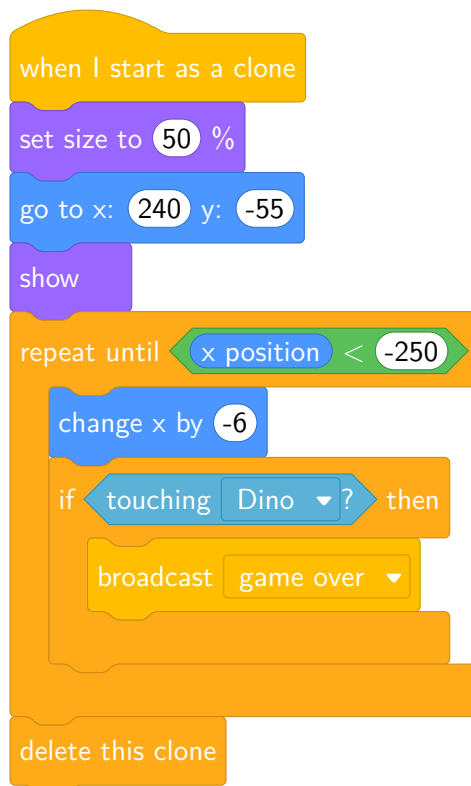
`stop [all]`

Halt every script in the project - the score tick and all the clones included. Used last, after the game-over message is shown.

Build it.

1. Back to the **cactus clone**. Fill the empty `if <touching [Dino]?>` with one block: `broadcast [game over]`. Make the message - click the dropdown, *New message*, name it *game over*. Now a crash shouts to the whole project.
2. The Dino listens. On the **Dino** add a new script: `when I receive [game over]`, then `say (join (Game over! Score: ) (score))` to show the final score, and finally `stop [all]`.
3. `stop [all]` is the *last* block on purpose: it freezes the score tick and removes every cactus and cloud clone - but only after the Dino has shown the game-over message, so the message is not cut off.
4. Green flag. Survive and your score climbs; the first cactus that touches the Dino now broadcasts *game over*, the Dino shows your total, and everything halts. The game is complete.

The two halves that close the loop: the cactus's crash now broadcasts, and the Dino receives it:



Cactus clone, completed: a crash now broadcasts game over



Dino: on game over, show the score, then stop everything

This is the payoff. **broadcast** is a one-to-many shout: the cactus fires game over and moves on; every **when I receive [game over]** in the project runs at once - no sprite has to poll or watch a variable. Here only the Dino listens, but you could add **when I receive [game over]** to the clouds to freeze them, to a sound sprite to play a thud, to a Game Over banner to show itself - all from the single message. **stop [all]** comes last so the message is delivered and drawn before everything halts. (There is also **broadcast and wait**, which pauses the sender until listeners finish - not needed here.)

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Add a surprised look: give the Dino a **dino-hit** costume and, at the top of the **game over** script, **switch costume to [dino-hit]** before the **say**.
2. Freeze the sky too: add **when I receive [game over]** to the **Cloud** with **stop [other scripts in sprite]**, so the clouds stop on the same message - one signal, many reactions.
3. Keep a high score: make a **best** variable, and on **game over** **if <(score) > (best)>** then **set [best] to (score)**.

Expected: The best run so far is remembered across games.
